

Platform Rebuild Proposal — Executive Summary

Purpose: Convince management to approve an incremental platform rebuild that reduces version sprawl, speeds delivery, and makes client customization and integrations sustainable.

Audience: Executive leadership, product, engineering, professional services, support.

Date: June 2026

1. Core recommendation

Rebuild the platform **incrementally** using a **strangler-fig** approach (Section 2): a single canonical core with **versioned extension and integration packs** — not a big-bang rewrite and not continued forking per client.

Critical constraint: the current application embeds years of domain knowledge that is **poorly documented** except through extensive unit and integration tests. The rebuild must **preserve as much of that knowledge as possible** by targeting a **new core API** with an **adapter layer** that delegates to legacy behaviour until parity is proven — not by discarding working code and rediscovering edge cases from scratch (Section 9).

Ask:

- Charter a **Platform 2.0** program with dedicated capacity (not 20% time).
- Approve a **3–6 month foundation + pilot** phase with measurable exit criteria.
- Adopt policy: **no new git submodules, no new core forks, no new SaveChanges workflow handlers.**

2. The strangler pattern: how we rebuild without a big-bang

Named after the strangler fig — a vine that grows around a host tree and eventually replaces it — the **strangler pattern** means building the new platform **alongside** the legacy application and **moving traffic one slice at a time**, not replacing everything in a single rewrite.

```
Start:      Users → Legacy application (everything)

Mid-way:    Users → Router / new API
               |
               |→ New import path
               |→ Legacy (planning, hours, UI, ...)
```

End: Users → New platform
 Legacy → retired

Per domain: pick a slice → build behind a clear boundary → route only that slice → prove parity (tests, pilot) → retire legacy path → repeat.

Big-bang rewrite	Strangler pattern
High delivery and revenue risk	Lower risk per step
Long period with little shippable value	Value after each slice
Easy to lose tacit domain behaviour	Legacy keeps running; tests guard parity
Hard to roll back	Route traffic back if a slice fails

Layer	Strangler mechanism
API & domain logic	New core API + adapter layer (Section 9)
Workflows	Retire handlers and Hangfire jobs one family at a time
Frontend	New SPA + versioned API; replace Razor islands (Section 10)
Customizations	Tenant packs on new core — not new submodules

Main risk: “two systems forever” — mitigated by domain sunset dates and adapter exit criteria (Section 17).

One line: *Build the new platform around the old one, move one piece at a time, and remove each legacy piece only when the replacement is proven equivalent.*

3. The problem (in business terms)

We do not have a feature problem. We have a **variant problem**.

Symptom	Business impact
Simple changes take weeks	Missed commitments, unhappy customers
Regressions in unrelated areas	Support load, emergency releases
Integrations behave differently per client	High PS cost, slow onboarding
Cloud + on-prem + customized builds	Too many versions to test and support
“Which version is running?”	Long incidents, risky upgrades

New hires slow to contribute	Delivery does not scale with headcount
High cognitive load per change; long code reviews	Low throughput, senior time in archaeology, team fatigue
UI layers accumulated without a “start over” boundary	Slow UI delivery, inconsistent UX, no reliable path to Cypress / Playwright / Selenium

One line: *We are paying a growing tax on every feature because customization and integrations were implemented as compile-time forks, not runtime configuration.*

Frontend: if we could do it over

If we could start the UI again, three decisions would change on day one — even though the original approach was rational at the time.

We began with a then-new Visual Studio template: Razor (.cshtml), npm libraries, and AngularJS in **one ASP.NET project**. That was genuinely productive — one build, shared bundling, straightforward local development. The mistake was not the template; it was **not drawing a hard line** when the product outgrew it.

Each later shortcut felt justified under delivery pressure: Razor pages for quick screens, Web API when features needed JSON, Vue (alongside leftover AngularJS) for richer client behaviour. Nobody set out to build a hybrid. We **accumulated** one — without a versioned API contract, without one client framework, and without a surface stable enough for automated UI tests.

What we would do differently:

1. **Separate backend and frontend from the start** — ASP.NET as API and auth host; the UI as its own SPA with its own build and release cadence.
2. **Pick one SPA philosophy — Angular or Vue**, not both, and not Razor for core product screens.
3. **Define the API first** — documented, versioned OpenAPI as the only boundary — so Cypress, Playwright, or Selenium can test critical flows without depending on server-rendered markup.

We cannot rewind. We *can* stop extending the hybrid and migrate toward what we would have built if we had known where the product was going. That counterfactual is the frontend strand of Platform 2.0 (Section 10).

Architecture exceeded its intended envelope

The original **layered architecture** was a reasonable choice for a smaller product with narrower client differences. It has since **expanded beyond what it was designed to carry** (see Section 4 for the typical request and save path). Client-specific behavior now lives in **git submodules**, service inheritance, and schema forks — so “the product” is effectively many variants sharing a name. Features such as **workflows** and **Hangfire** added a second, implicit orchestration layer on top of EF change handlers. We are not struggling because the domain is inherently impossible; we are carrying **two kinds of diversity at once** — a rich project-management domain *and* per-client implementation diversity — without a clear boundary between platform and customization.

Those choices were rational under past delivery pressure. The toll is visible now: every change requires reconstructing hidden context before it can be implemented or reviewed safely.

Developer productivity and cognitive load

The highest day-to-day cost on this codebase is often not missing features — it is **cognitive load**: how much context an engineer must hold to change one thing without breaking another. A small PR may require understanding tenant-specific branches, handler chains, background jobs, and data-model variants before anyone can judge risk. That shows up as **long code reviews**, slow iteration, and reluctance to touch areas that look unrelated.

This is a **delivery and retention** issue, not a preference for greenfield work. The ask is not to throw away what we have. It is to **stop extending the submodule-and-handler model** and fund incremental simplification — one domain at a time, with parity tests and a pilot — so that most engineering energy goes into the domain problem, not archaeology.

Embedded knowledge vs. architectural baggage

A rebuild carries a real risk: **losing knowledge** baked into the current application — edge cases, client rules, integration behaviour, and planning semantics that accumulated over years of production use. That knowledge is **rarely written down**; it lives in code paths, in engineers’ heads, and most reliably in our **numerous unit and integration tests**.

At the same time, the **structures that hold that knowledge** — submodules, handler chains, service inheritance, hybrid UI — are the baggage dragging delivery down. We cannot keep everything as-is, and we must not big-bang rewrite and hope we remember what mattered.

Goal: preserve behaviour, shed structure. Target a **new core API** and route traffic through an **adapter layer** that delegates to legacy until each domain is understood, tested, and ported (Section 9).

4. Legacy application layers: a typical save path

The current product follows a **classic layered stack** on paper — but a single user action can traverse every layer and still trigger **hidden work** after the HTTP response returns.

Request path (synchronous)

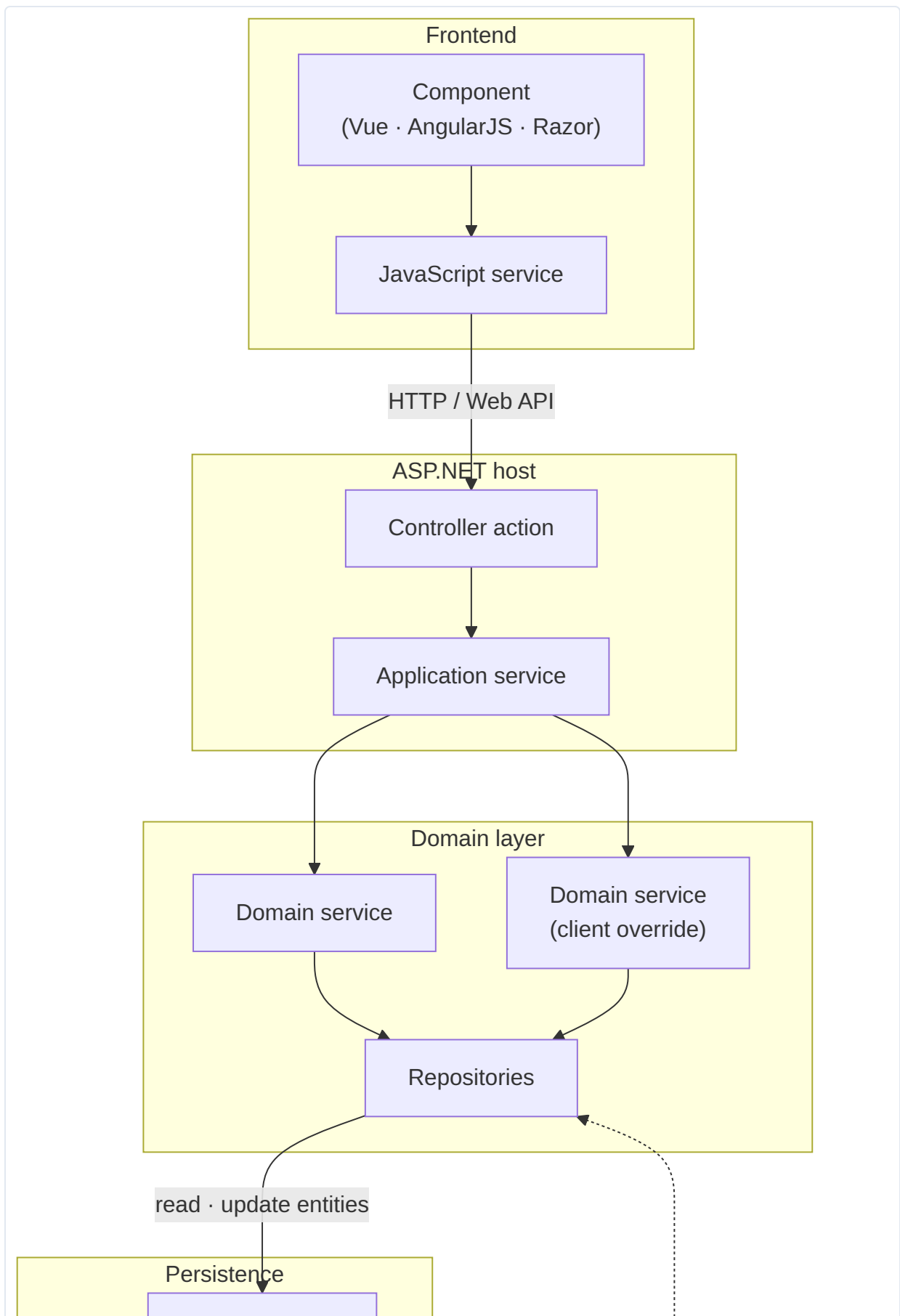
A typical use case begins in a **frontend component** (Vue, legacy AngularJS, or Razor-hosted script). The component calls a **JavaScript service**, which issues HTTP to a **controller action**. The controller delegates to an **application service**, which calls one or more **domain services** (often with client-specific subclass overrides). Domain services use **repositories** to read and update **entities** via Entity Framework. **SaveChanges** persists to SQL Server — but that is not where the work ends.

Side effects on save

EF change handlers fire on **SaveChanges** — implementing rollups, integration side effects, validation, and denormalization (legacy of Access / stored procedures). Handlers cause **additional reads and writes** (sometimes nested **SaveChanges**) and can **enqueue Hangfire jobs** or workflow steps that run after the HTTP response.

Background processors

Background jobs often run **custom SQL** outside the repository/entity path — a third read/write surface alongside handlers. Together, handlers and background jobs form a **parallel workflow system** on top of the layered stack (Section 5).



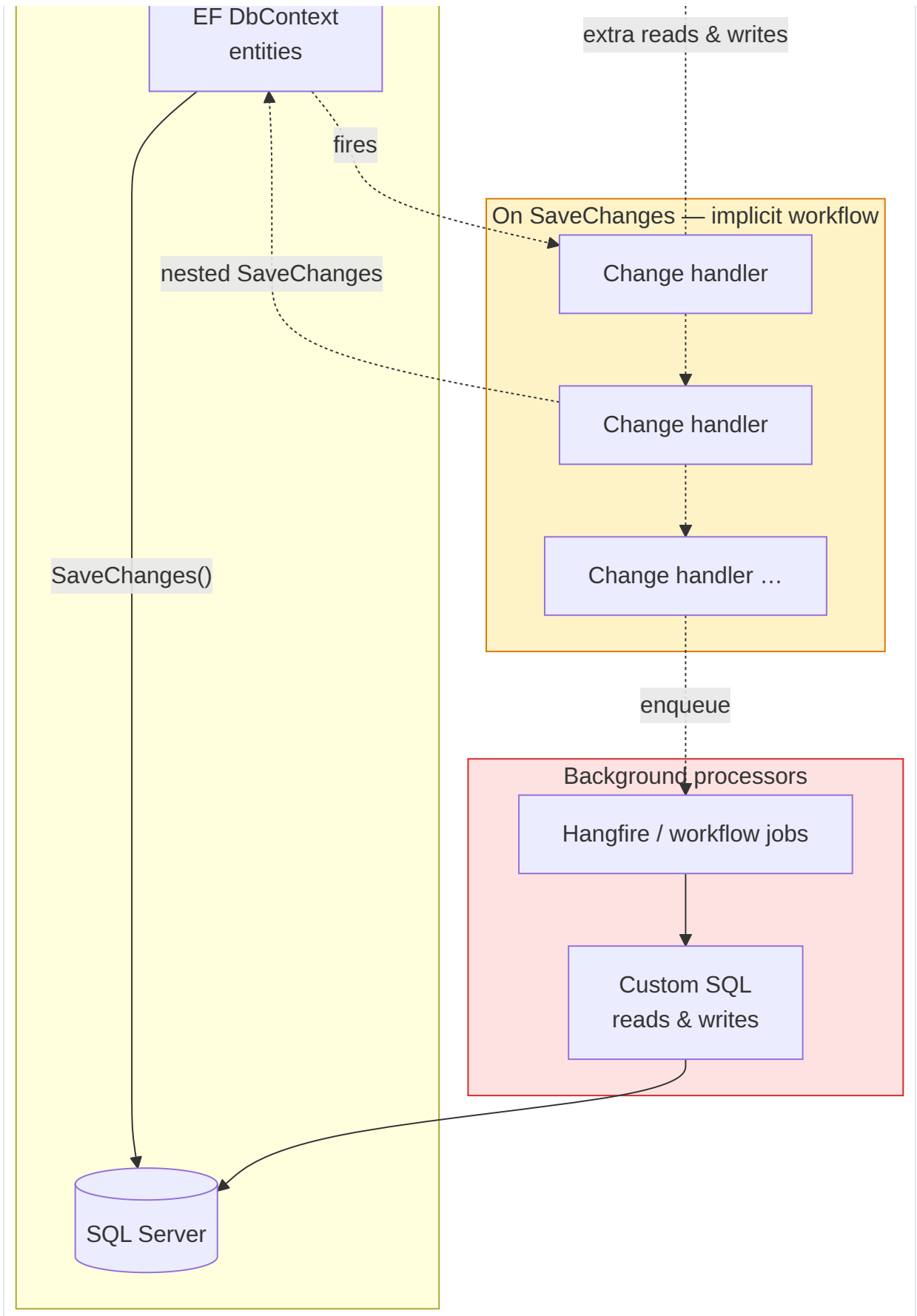


Figure 1 — Typical save path: layered call stack plus implicit handler chain and background SQL

One line: *A save in the old application is a layered call stack plus an unpredictable handler chain and optional background SQL — not a single transaction boundary.*

5. Root technical causes

The stack reflects **scope creep on the architecture**: patterns that worked at one scale now compound each other.

1. **Git submodules and client-specific code branches** — the decision to customize per client via submodules made sense early on; it now multiplies release combinations and merge cost.
2. **Class inheritance for services** spreads logic across opaque override trees within an already layered stack.
3. **Per-client data model forks** make migrations, APIs, and reporting unsustainable.
4. **EF SaveChanges change handlers** implement hidden workflows (legacy of Access DB + stored procedures).
5. **Hangfire jobs and workflow features** duplicate and defer logic — a **parallel workflow system** alongside handlers, harder to trace than either alone.
6. **Weak domain boundaries** — WBS, planning, hours, imports, and integrations are coupled while client variance cuts across every layer.
7. **Frontend accumulated without a boundary** — each layer solved a near-term need; we never applied the “start over” rule: one SPA, one versioned API, tested independently.

This is normal debt from years of shipping under pressure. It is not a people problem — and it is **not an argument for a big-bang rewrite**. It is an argument for a controlled boundary between core and customization going forward.

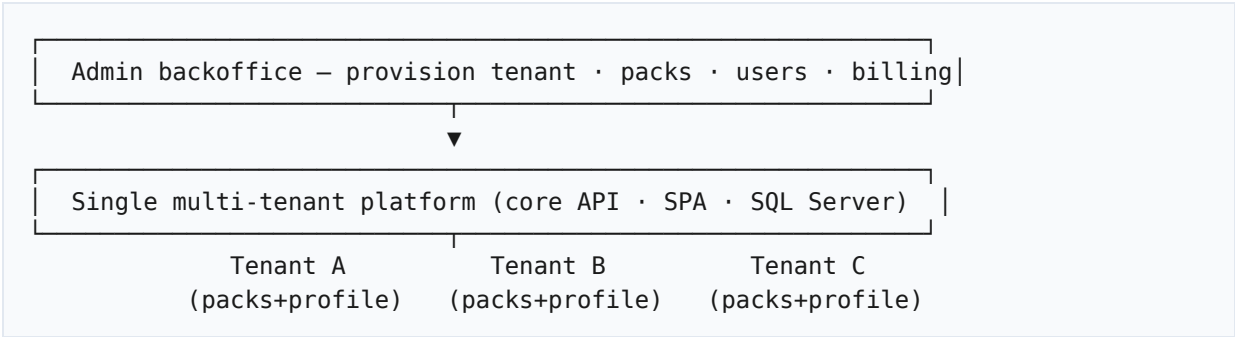
6. Strategic goal: one platform, many configurations

Dimension	Target
Deployment	Same product artifact for cloud SaaS and on-prem (profile-driven config).
Integration	Connector packs (SAP, PLM, HR, file, etc.) — not core forks.
Customization	Configuration, policy hooks, and extension actors — not submodules.

Success criterion: A new client integration or customization ships as a **versioned pack** enabled per tenant, not as a separate product build.

End state: one multi-tenant platform

The **end stage** is a **single multi-tenant platform** — one canonical cloud deployment where every client is a **tenant** (TenantId isolation; dedicated SQL Server only for enterprise tier). Behaviour differences are **enabled packs and profile settings**, not submodules or custom builds. **Onboarding a new client is an operational action**, not a development project.



Admin backoffice

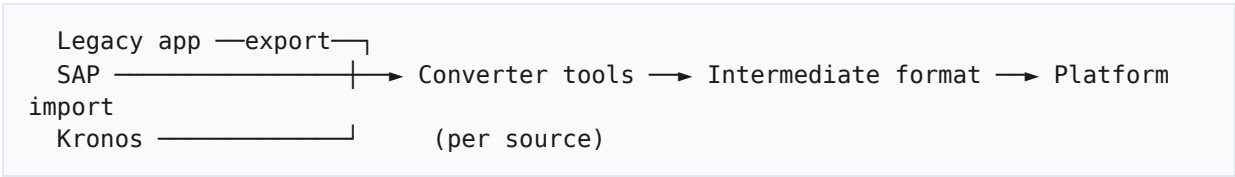
Separate from the end-user product UI — the **control plane** for platform operators:

- **Tenant provisioning** — name, subdomain, region, performance tier
- **Pack management** — enable integration and customization packs per tenant
- **Tenant profile** — SSO/AD, cloud vs on-prem profile, feature flags
- **User bootstrap** — initial admin, seats, role templates
- **Billing** (target) — tier, seats, usage meters, invoice or payment-provider hooks

Target: “Add client” in backoffice → tenant live on shared platform in hours, not weeks of branching.

Project portability & onboarding hub: intermediate exchange format

One **versioned intermediate format** is the canonical interchange for project onboarding — legacy export, **SAP, Kronos**, and future systems all converge on it; one **platform import** pipeline consumes it.



Component	Role
Legacy export	Export projects from current product
Integration converters	Map SAP, Kronos, etc. into intermediate format — integration pack tooling

Intermediate format	Versioned schema; single front door for onboarding data
Platform import	Import API / actors; idempotent; dry-run + validation

Onboarding: provision tenant → run converter (or legacy export) → import — same playbook, not a bespoke pipeline per system. Converters can start file-based and grow into scheduled connectors emitting the same format.

Journey from today to end state

Stage	Milestone
Today	Variants, submodules, hybrid UI, per-client builds
1 — Foundation	Versioned API, adapters, tenant profile; freeze forks
2 — First tenant	Pilot on new API; export/import bridge on real projects; one legacy path retired
3 — Domain expansion	Core domains + SPA; migration runbook; SAP / Kronos converters
4 — Pack-only custom	All new variance as versioned packs
5 — Operations plane	Admin backoffice MVP: provision tenant without code
6 — End state	Single platform; new client via backoffice + billing; legacy gone

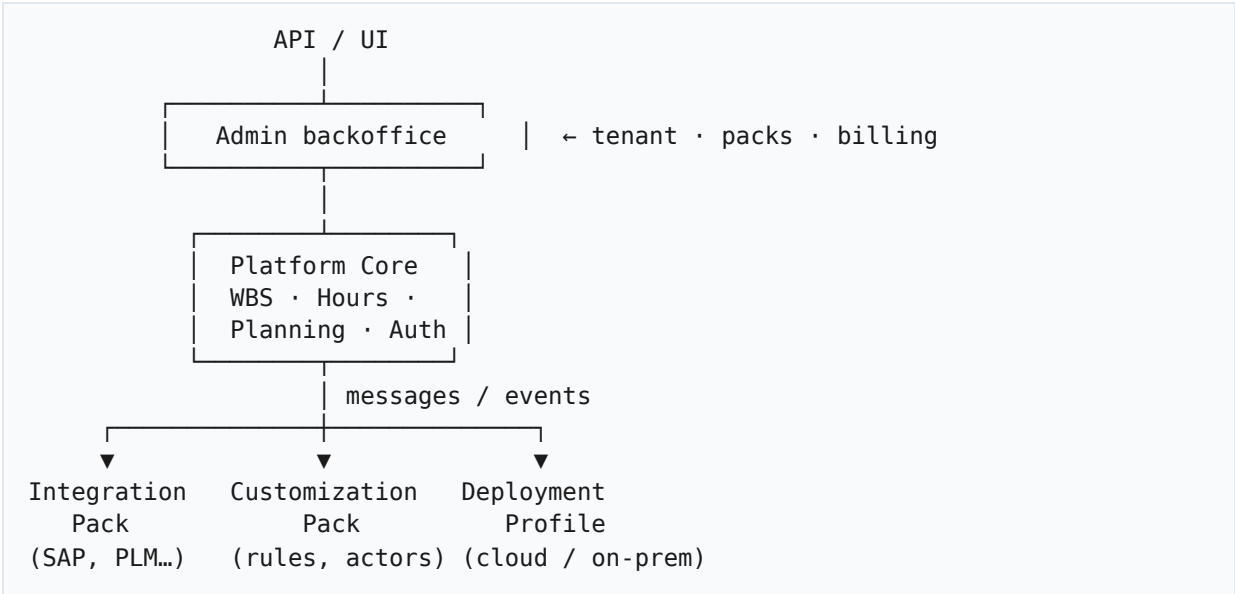
Stages 1–4: technical strangler. Stages 5–6: operational maturity — **operate and sell**, not **build per client**. Section 15 maps phases to these stages.

7. Why the current customization model fails

Approach	Long-term result
Git submodules per client/integration	Version matrix explosion, merge hell
Subclassing services	Fragile base classes, hard-to-find overrides
Custom EF models per client	Forked migrations, APIs, reports, upgrades
if (tenant == X) in core	Accidental complexity, untestable branches

Replacement principle: *Professional services may customize behavior, not the core release artifact.*

8. Target architecture: platform core + packs



Core owns: canonical domain, invariants, APIs, persistence boundaries, external ID registry.

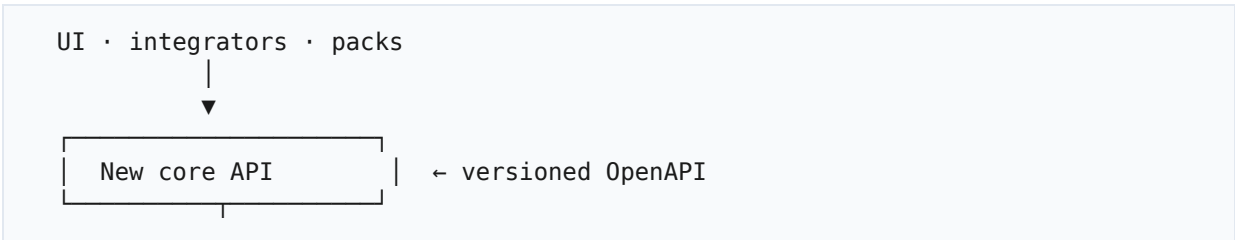
Packs own: client- or system-specific mapping, validation, enrichment, connectors, and policy steps.

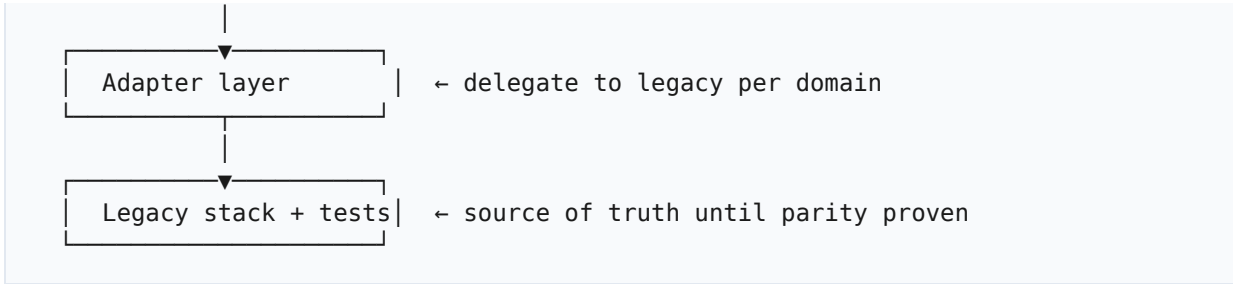
9. Preserve embedded knowledge: new core API + adapter layer

Rebuild is not permission to forget what the product already knows — but staying on the current stack is not neutral either.

Preserve (asset)	Shed (baggage)
Behaviour in unit & integration tests	Submodules and per-client forks
Production-proven edge cases	Implicit SaveChanges handler chains
Working integration mappings	Service inheritance trees

Much of what makes the product valuable is **tacit** — poorly documented outside code and tests. A greenfield rewrite risks missing behaviour tests currently guard.





How adapters preserve knowledge: delegate first; use existing tests as **parity gates**; extract logic into the new core only after equivalence is proven; document at the API boundary; sunset every adapter with explicit exit criteria.

10. Frontend: applying the “do it over” decisions

The backend rebuild only delivers half the value if we keep adding to the hybrid. This section turns the Section 3 counterfactual into concrete architecture and migration.

What we started with (and why it worked)

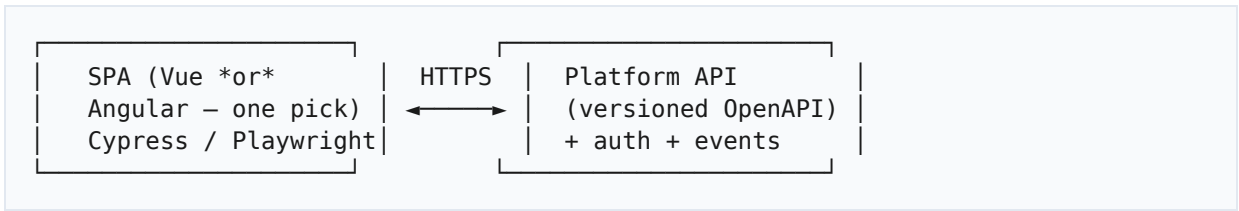
Starting point	Why it was the right call then
Visual Studio ASP.NET template	Razor + npm + AngularJS in one project
Single solution	One build, shared bundling, fast local dev

What accumulated (without a plan)

Layer added	Typical trigger	Cost in hindsight
Razor pages	Fast server-rendered screen	Second rendering model
Web API	JSON for AJAX / mobile	No published contract
Vue (+ legacy AngularJS)	Richer client UX	Two SPA philosophies

The result: duplicated auth and routing, inconsistent UX, and no stable target for Cypress, Playwright, or Selenium.

What we would build instead (target)



Principle	Requirement
Separation	Frontend and backend are separate build and deploy units; no business logic in Razor except transitional shells during migration.
Single framework	One SPA stack for new work — Angular or Vue , decided in Phase 0; the other is sunset on a published schedule.
API as contract	OpenAPI per major version; breaking changes require version bump and migration notes.
Testability	Critical flows covered by UI automation (Cypress, Playwright, or Selenium) against the versioned API — not against server-rendered markup.

How we get there from here: freeze new Razor features and second SPA framework; expose API first; replace Razor islands with SPA routes behind feature flags; retire co-located client bundles as each domain moves.

11. Akka.NET: why it fits (orchestration, not fashion)

Akka.NET is proposed because our hardest problems are **workflows**, not CRUD: import/export pipelines, integration retries, long-running recalculations, and tenant-specific processing steps.

Actors vs. inheritance / submodules

Legacy	Akka approach
AcmeImportService : ImportService	ImportPipeline → [MapActor, AcmeRulesActor, PersistActor]
Client submodule	Tenant profile registers actor pack
Hidden handler chain	Explicit message pipeline with correlation ID

Actors vs. SaveChanges handlers + Hangfire

Legacy pattern	Problem	Akka replacement
EF change handlers on save	Implicit order, nested saves, hard to test	Command → orchestrator actor → persist actor
Hangfire jobs with business logic	Two workflow systems, duplicated rules	Supervised connector/workflow actors
Scheduled sync	Job-centric observability	Scheduler triggers actor message

Rule: SaveChanges is for **persistence**, not for **business process orchestration**.

```
Command (ImportProject, BookHours, RecalculatePlan)
  → Orchestrator actor
    → Persist actor (sole EF boundary for that workflow)
    → Integration / rollup / notification actors
    → Domain events (ImportPersisted, PlanRecalculated)
```

What Akka is NOT for: simple CRUD, DbContext in every actor, or replacing a disciplined domain model. Use actors at **boundaries**.

12. Data model strategy (avoid per-client schema forks)

1. **One canonical schema** for all customers on **SQL Server** (production and on-prem standard).
2. **Governed extension data** for rare attributes (registry + validation).
3. **External ID mapping** — source systems map to internal entities without alternate keys.
4. **Read models / projections** for client-specific reporting.

Customize **behavior and mappings**, not the physics of the database except through extension points.

Datastore: SQL Server only — no SQLite or per-client schema forks. Shared-database tenants use TenantId + indexes; enterprise tenants may use dedicated SQL Server instances (same schema, different connection).

13. Cloud and on-prem

Same binary; differences are **configuration and enabled packs** (identity, messaging, storage, updates, offline file import/export).

14. Proof of concept

An internal POC demonstrates: external IDs, actor-orchestrated persistence, template-first ordering, planning with FS/SS/FF/SF + lag, domain value types, and automated tests — including the **platform import** side of the export → intermediate format → import bridge (Section 6).

Message: We are asking to scale an approach that already produced working vertical slices.

15. Phased migration

Phases align with **journey stages** in Section 6 (Today → End state).

- **Phase 0** (Stage 1 prep): Inventory, adapter map, backoffice scope, **intermediate format** schema; SAP/Kronos converter scope.
- **Phase 1** (Stage 1): API, adapters, tenant profile; **platform import** from intermediate format (dry-run).
- **Phase 2** (Stage 2): Pilot tenant; **legacy export** + import round-trip on real projects.
- **Phase 3** (Stages 3–4): Expand domains, SPA, packs; migration runbook; **SAP / Kronos converters** → intermediate format.
- **Phase 4** (Stage 5): **Admin backoffice MVP** — provision tenant, packs, users; optional import job status.
- **Phase 5** (Stage 6): Retire legacy; billing; onboard clients via backoffice (< 1 business day SLO).

Hangfire may remain temporarily as a **scheduler trigger** only during migration.

16. Metrics

Track: active forks, lead time, change failure rate, time to add integration, “unknown version” tickets, on-prem upgrade rate, % work delivered as packs vs forks, **median PR review time, context switches per feature, % core screens on versioned API + single SPA, critical-path UI scenarios in E2E suite, % API traffic through adapters vs native new core, parity test pass rate, time to provision tenant via backoffice, active client-specific core branches (target: 0), and projects migrated via export/import bridge.**

17. Risks

Risk	Mitigation
Loss of tacit domain knowledge	Adapter layer delegates to legacy; existing tests as parity gates
Adapter layer becomes second monolith	Per-domain adapters with sunset criteria; thin delegation only
Two systems forever	Domain sunset dates; strangler per module; adapter exit criteria
Akka learning curve	Training, narrow actor boundaries, message contract tests
UI rewrite scope creep	Strangler per screen; API-first; one framework only

Big-bang temptation	Explicit anti-big-bang charter; pilot-first
---------------------	---

18. Options

Option	Verdict
A. Status quo	Cheapest now; cost accelerates
B. Incremental rebuild (recommended)	Balanced risk; continuous delivery
C. Big-bang rewrite	Highest revenue and execution risk

19. Executive summary

*Our product's complexity is driven less by feature count than by years of client-specific customizations and integrations, delivered through git submodules, service inheritance, customized data models, EF change handlers, and Hangfire jobs — patterns inherited from an earlier Access/stored-procedure architecture. The **layered design and submodule-based customization** were fit for an earlier scope; the product has outgrown that envelope. Workflows and background jobs added implicit orchestration on top of an already diverse domain and diverse clients. On the **frontend**, we would now treat UI the same way: **if we could do it over**, we would never have let Razor, Web API, Vue, and legacy AngularJS share product screens without a versioned API — the lesson we apply in Platform 2.0. Combined with cloud and on-prem delivery, this created too many de facto product versions to test, upgrade, and support — and a **high cognitive load** that slows delivery and lengthens code review even for experienced engineers.*

*The proposed rebuild shifts to a **platform-and-packs architecture** whose **end state** is a **single multi-tenant platform**: new clients provisioned through an **admin backoffice** (packs, profile, users, billing over time) — not forks or bespoke builds. We **preserve embedded domain knowledge** via a **versioned core API** and **adapter layer**, following the **strangler pattern** through defined journey stages. On the UI: **separate SPA, versioned API, automated UI tests**. Long-running work moves into **explicit Akka.NET workflows**.*

*This restores predictable delivery, reduces support matrix size, protects professional services margin, and gives customers a credible upgrade path without forking the product for each engagement — **without** a big-bang rewrite.*

20. Performance and large projects (40–50,000 activities)

Customers range from small projects (~40 activities) to large programmes (~50,000 activities). Responsiveness is a **product architecture** requirement, not only a hardware decision.

Design principle

Interactive paths stay small; heavy work runs asynchronously on bounded slices.

Path	Target	Pattern
Open project / browse WBS	< 500 ms	Paginate, lazy-load tree, summaries
Edit one activity	< 100 ms	Targeted read/write
Full plan recalculation (large project)	Seconds, background	Planning actor + progress events
Gantt view	Smooth scroll	Virtualized rows + visible time window
Bulk import	Minutes OK	Actor pipeline + batched upsert + progress

Database (SQL Server)

- **Never load full project tree by default** — lazy expansion, cursor pagination, search endpoints.
- **Indexes:** (TenantId, ProjectId), (ComponentId), (ActivityId), assignment and relation FKs.
- **Component-level rollups** — cached budgeted hours, % complete, date ranges.
- **Dedicated SQL Server** for large enterprise tenants (isolation tier).

API tiers

Endpoint style	Purpose
/projects/{id}/summary	Counts, dates, rollups only
/components?parentId=	One tree level at a time
/activities?projectId=&cursor=	Paginated flat list / search
/projects/{id}/plan?from=&to=	Windowed Gantt data

Return **projections**, not full entity graphs, to the UI.

Planning at scale

- **Store schedule snapshots** — UI reads persisted plan; does not recalculate on every page load.
- **Async recalculation** via `PlanningCoordinatorActor` for projects above threshold (e.g. 500 activities).
- **Incremental scheduling** — on single edit, recalculate affected subgraph only; full recompute on import or bulk relation change.
- **Never block HTTP** on full 50k-activity forward pass.

UI

- Virtual scrolling for Gantt and activity lists.
- Time-windowed Gantt (load visible months, fetch more on pan).
- Component-level aggregation with expand-for-detail.
- Debounced plan triggers (one background job per burst of edits).

Imports

- Batched upsert (e.g. 500 rows per transaction) through import actor pipeline.
- Progress events (`ImportProgressUpdated`) for UI feedback.
- Idempotent external IDs for safe retry.

Multi-tenant isolation

- **Performance tiers** in tenant profile: standard vs large.
- Rate limits and fair queuing for heavy jobs (import, replan).
- Large tenants: dedicated DB + worker capacity.

SLOs (proposal targets)

- Up to **500 activities**: interactive edits < 1 second.
- Up to **50,000 activities**: browse and edit remain interactive; full replan in background with progress within minutes.
- Load-test gates at **40, 500, 5,000, and 50,000** activities before major client migrations.

21. Decision requested

1. Approve **Platform 2.0** program charter and dedicated team.
2. Fund **discovery + foundation + one pilot domain**.

3. Enforce **no new submodules / handler workflows**.
4. Standardize on **SQL Server** for cloud and on-prem data stores.
5. Adopt **performance SLOs** for large projects (Section 20).
6. Standardize the **frontend** using the “do it over” model: one SPA framework and a **versioned public API** (Section 10).
7. Mandate **adapter-first migration** to the new core API so existing tests preserve domain knowledge (Section 9).
8. Approve **admin backoffice** roadmap (Phase 4) as the path to onboard tenants without forks.
9. Adopt **intermediate exchange format** as the contract for **all project onboarding** — legacy, SAP, Kronos, and future integrations (Section 6).
10. Name **executive sponsor** (engineering + product + commercial).
11. Select **pilot customer** or flagship project for first cutover **and first export/import migration**.

Document generated from engineering strategy discussions. Internal use.